

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа номер 5

Выполнил:

Бочаров Евгений

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках данной работы необходимо:

- Изучить основные команды пакетного менеджера NPM (установка, инициализация, управление зависимостями).
- Инициализировать новый проект на Vue 3 с использованием Vue CLI.
- Настроить подключение внешних библиотек: Axios (HTTP-запросы), Pinia (state management), Bootstrap (UI-стили).
- Реализовать простое приложение «Заметки» с возможностью:
- Просмотра списка заметок (получение данных с API)
- Создания новой заметки (отправка данных на API)
- Отображения данных в виде карточек с использованием компонентного подхода
- Настроить роутинг и структуру проекта согласно принципам компонентной архитектуры.

Ход работы

1. Инициализация проекта
 - Установлен Node.js и npm (LTS-версии)
 - Создан проект командой `npm init vue@latest` с подключением Vue Router и Pinia
 - Настроен скрипт запуска `npm start` с портом 8080 в `package.json`
2. Установка зависимостей
 - Установлены пакеты: `axios`, `pinia-persists`, `bootstrap`, `@popperjs/core`
 - Решены конфликты версий зависимостей с помощью флага `--legacy-peer-deps`
3. Настройка структуры проекта
 - Созданы модули API: `instance.js`, `notes.js`, `index.js` для работы с JSON Server
 - Настроен Pinia store (`stores/notes.js`) с действиями `loadNotes` и `createNote`
 - Реализованы компоненты: `NoteCard.vue` (карточка заметки), `BaseLayout.vue` (лейаут), `NotesPage.vue` (страница со списком и формой)
4. Реализация функционала

- Настроена двусторонняя привязка данных через v-model в форме создания заметки
- Реализована загрузка заметок при монтировании компонента (mounted hook)
- Добавлена обработка отправки формы с обновлением списка после создания

5. Запуск и тестирование

- Запущен бэкенд на JSON Server (json-server db.json -m ./node_modules/json-server-auth)
- Запущен фронтенд (npm start), проверена работа CRUD-операций в браузере

Вывод

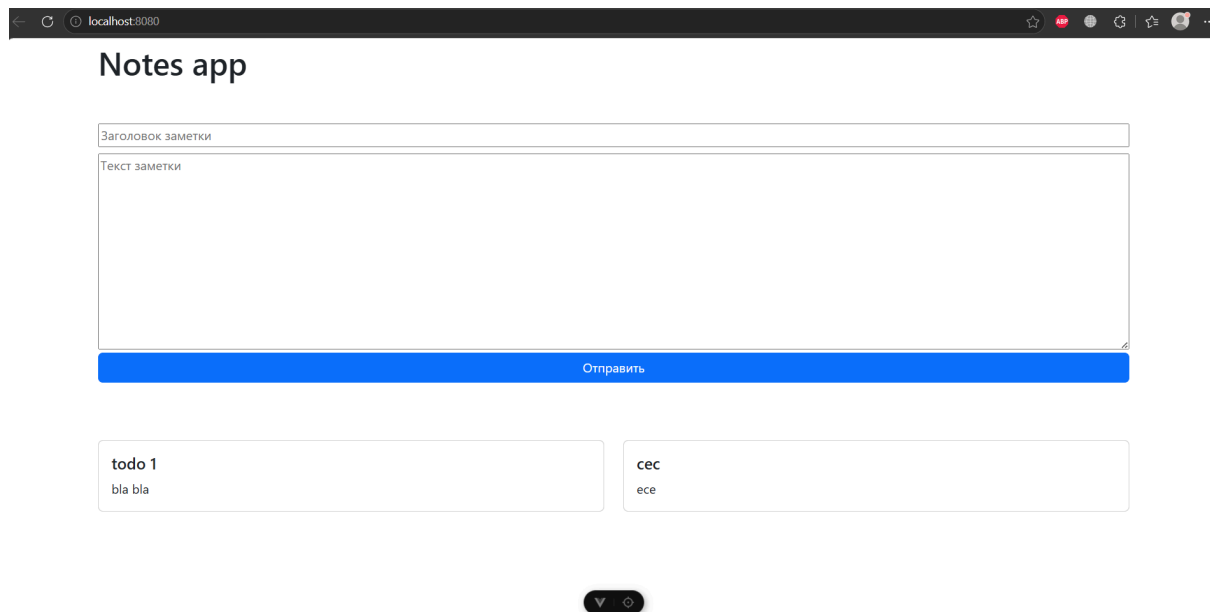
В ходе выполнения работы были освоены ключевые навыки работы с экосистемой Vue.js:

- Научился инициализировать и настраивать Vue-проекты через CLI
- Освоил базовые команды npm: install, init, управление package.json
- Разобрался с компонентным подходом, передачей данных через props и v-model
- Научился интегрировать внешние библиотеки (Axios, Pinia, Bootstrap) в проект
- Реализовал взаимодействие с внешним API и управление состоянием приложения

Приложение «Заметки» полностью функционально: позволяет просматривать, создавать и отображать заметки с сохранением данных через JSON Server. Полученный опыт закладывает основу для разработки более сложных одностраничных приложений (SPA) на Vue.js.

Приложения

скриншоты готового приложения



КОД

backend/db.json

```
{  
  
  "users": [  
  
    {  
  
      "email": "mail@example.com",  
  
      "password":  
"$2a$10$YUqF5kLaSlma7IcFNrcKhu9Yy4s3S3tw02T28JsUzbiZk  
37myZIfK",  
  
      "firstName": "John",  
  
      "lastName": "Doe",  
  
      "id": 1  
    }  
  ],  
  
  "notes": [  
  
    {  
  

```

```
    "id": 1,

    "name": "todo 1",

    "tags": [

        1

    ],

    "text": "bla bla",

    "userId": 1
},
{

    "name": "cec",

    "text": "ece",

    "userId": 1,

    "id": 2

}
],
"tags": [

    {

        "id": 1,

        "name": "to-do"

    }

]
}
```

backend/package.json

```
{  
  
  "name": "backend",  
  
  "version": "1.0.0",  
  
  "scripts": {  
  
    "start": "json-server db.json -m  
./node_modules/json-server-auth"  
  
  },  
  
  "devDependencies": {  
  
    "json-server": "^0.17.0",  
  
    "json-server-auth": "^2.1.0"  
  
  }  
}
```

vue-project/src/api/[index.js](#)

```
import instance from '@api/instance'  
  
import NotesApi from '@api/notes'  
  
const notesApi = new NotesApi(instance)  
  
export {  
  
  notesApi  
}
```

vue-project/src/api/[instance.js](#)

```
import axios from 'axios'

const apiURL = 'http://localhost:3000'

const instance = axios.create({
  baseURL: apiURL
})

export default instance
```

vue-project/src/api/[notes.js](#)

```
class NotesApi {

  constructor(instance) {

    this.API = instance

  }

  getAll = async () => {

    return this.API({

      url: '/notes'

    })

  }

  createNote = async (data) => {

    return this.API({

      method: 'POST',
```

```

        url: '/notes',
        data,
        headers: {
            'Content-Type': 'application/json'
        }
    })
}
}

export default NotesApi

```

vue-project/src/components/NoteCard.vue

```

<template>

  <div class="card">

    <div class="card-body">

      <h5 class="card-title">{{ name }}</h5>

      <p class="card-text">{{ text }}</p>

    </div>

  </div>

</template>

<script>

export default {

  name: 'NoteCard',

```



```

props: {
  name: {
    type: String,
    required: true
  },
  text: {
    type: String,
    required: false
  }
}
}
</script>

```

vue-project/src/layouts/BaseLayout.vue

```

<template>
  <main class="container my-2">
    <slot/>
  </main>
</template>

```

vue-project/src/router/[index.js](#)

```

import { createRouter, createWebHistory } from
'vue-router'

const router = createRouter({

```

```

history: createWebHistory(import.meta.env.BASE_URL),
routes: [
  {
    path: '/',
    name: 'notes',
    component: () =>
import('../views/NotesPage.vue')
  }
]
}))

export default router

```

vue-project/src/stores/[counter.js](#)

```

import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter',
() => {

  const count = ref(0)

  const doubleCount = computed(() => count.value * 2)

  function increment() {

    count.value++

  }
}

```

```
    return { count, doubleCount, increment }
  })
}
```

vue-project/src/stores/[index.js](#)

```
import { persist } from 'pinia-persists'

import { createPinia } from 'pinia'

const pinia = createPinia()

pinia.use(persist())

export default pinia
```

vue-project/src/stores/[notes.js](#)

```
import { defineStore } from 'pinia'

import { notesApi } from '@api'

const useNotesStore = defineStore('notes', {

  state: () => ({

    notes: []

  }),

  actions: {

    async loadNotes() {

      const response = await notesApi.getAll()

      this.notes = response.data

      return response
    }
  }
})
```

```

    },

    async createNote(data) {
        const response = await
notesApi.createNote(data)

        this.notes = response.data

        return response
    }
}

}))

export default useNotesStore

```

vue-project/src/views/NotesPage.vue

```

<template>

  <base-layout>

    <h1>Notes app</h1>


    <form
      ref="noteForm"

      @submit.prevent="createCard"

      class="d-flex flex-column my-5"
    >

      <input
        type="text"

```



```
</base-layout>

</template>

<script>

import BaseLayout from '@layouts/BaseLayout.vue'

import NoteCard from '@components/NoteCard.vue'

import { mapActions, mapState } from 'pinia'

import useNotesStore from '@stores/notes'

export default {

  name: 'NotesPage',

  components: { BaseLayout, NoteCard },

  data() {

    return {

      form: {

        name: '',

        text: '',

        userId: 1 // хардкод для теста

      }

    }

  },

  computed: {

    ...mapState(useNotesStore, ['notes'])

  }

}
```

```

    },
    methods: {
        ...mapActions(useNotesStore, ['loadNotes', 'createNote']),
        async createCard() {
            await this.createNote(this.form)
            await this.loadNotes()
            this.$refs.noteForm.reset()
        }
    },
    mounted() {
        this.loadNotes()
    }
}
</script>

```

vue-project/src/app.vue

```

<template>
  <router-view/>
</template>

<script>
export default {
  name: 'App'
}

```

```
}  
  
</script>
```

vue-project/src/[main.js](#)

```
import { createApp } from 'vue'  
  
import App from '@App.vue'  
  
import router from '@router'  
  
import store from '@stores'  
  
import 'bootstrap/dist/css/bootstrap.min.css'  
  
import 'bootstrap'  
  
import '@assets/main.css'  
  
const app = createApp(App)  
  
app.use(store)  
  
app.use(router)  
  
app.mount('#app')
```